

# 자기소개서

이기찬 | SRE / DevOps Engineer 지원

## 1. 커리어에서 추구하는 방향성

### "한 명이 만든 표준이 백 명을 편하게 하는 일"

저는 "코드를 짜는 사람"보다 "코드가 굴러가는 환경을 책임지는 사람"이 되고 싶습니다. 기능 하나가 잘 동작하는 것보다, 그 기능이 새벽 3시에도 누군가의 수동 개입 없이 안정적으로 살아 있는 상태가 훨씬 어렵고 가치 있는 일이라고 생각합니다.

Bithabit을 2년간 혼자 운영하며 이 감각이 단단해졌습니다. 처음에는 EC2 한 대에 docker-compose로 띄우는 식이었지만, 서비스가 14개로 늘어나자 "내 머릿속에만 있는 운영 지식"이 가장 큰 리스크라는 걸 깨달았습니다. 이후 모든 변경을 ArgoCD GitOps로 옮기고, AWS에서 OCI Ampere A1으로 인프라 전체를 이전하며 ARM64 멀티아키텍처 빌드까지 코드화했습니다. 결과는 비용 100% 절감보다도, "내가 잠들어도 시스템이 스스로 기록을 남기고 자기 상태를 설명해 주는 구조"를 갖추게 된 것이 더 큰 수확이었습니다.

그래서 다음 단계는 한 명의 운영자가 아니라 조직 전체의 운영 표준을 설계하는 일이라고 봅니다. 데브시스템즈 인프라셀이 "한 제품의 DevOps Best Practice를 넘어 여러 제품에 공통 적용 가능한 Best Practice를 고민한다"는 부분이 가장 깊이 와닿았습니다. 한 명이 만든 좋은 모듈이 다른 팀의 새벽을 구하는 일, 한 번 잘 잡힌 IaC가 신규 클러스터 프로비저닝을 30분 작업으로 끝내는 일—이런 레버리지가 큰 운영 엔지니어링이 제가 가고 싶은 방향입니다.

"코드는 짧게, 영향은 길게. 한 명이 만든 표준이 백 명의 새벽을 지킬 수 있도록."

## 2. 강점이 드러나는 영역

### (1) 모호한 장애를 단계별 절차로 시스템화하는 능력

Microsoft M365 기술지원에서 3개월간 Entra ID·하이브리드 클라우드 장애 200건 이상을 처리하며, 팀 20명 중 성공 해결률(SRR) 92.7%, 불만족(DSAT) 0%로 전 지표 1위를 기록했습니다. 이 숫자가 의미 있는 이유는 "기술 지식을 많이 안 사람이 1등 했다"가 아니라, "가장 모호한 장애를 가장 빨리 절차로 바꾼 사람이 1등 했다"는 점입니다.

예를 들어 이메일 도메인 전체 마이그레이션처럼 실패 시 데이터 손실로 직결되는 작업은, 고객의 모호한 불안("메일이 다 날아갈 것 같아요")을 그대로 받지 않고 "백업 → 이관 → 복구 → 검증" 4단계로 분해해 각 단계마다 롤백 포인트를 명시한 티켓 시스템으로 재구성했습니다. 이 방식이 데이터 손실 위험을 0%로 수렴시켰고, 동시에 다른 동료가 같은 유형의 케이스를 받았을 때 그대로 재사용할 수 있는 절차서가 되었습니다. 장애 대응을 "개인 영웅의 직관"이 아니라 "팀이 공유하는 표준 절차"로 만드는 일—이게 SRE의 포스트모템 문화와 같은 결이라고 생각합니다.

## (2) IaC와 GitOps로 인프라를 코드처럼 다루는 감각

Bithabit 인프라는 단일 노드 k3s 위에서 14개 서비스가 ArgoCD GitOps로 운영됩니다. 모든 매니페스트는 Git에 있고, 모든 변경은 PR로 들어옵니다. Traefik Ingress와 cert-manager를 연동해 Let's Encrypt Wildcard 인증서를 자동 발급·갱신하고, 내부 서비스는 사설 라우팅으로 분리해 외부 노출을 최소화했습니다.

이 작업을 하면서 IaC의 핵심은 "자동화"가 아니라 "재현 가능성"이라는 걸 배웠습니다. 새 노드에서 동일한 환경을 30분 안에 다시 세울 수 있는 상태, 누가 무엇을 언제 왜 바꿨는지 git log 한 줄로 설명되는 상태—이게 운영자가 휴가를 갈 수 있는 인프라의 최소 조건이라고 생각합니다. 데브시스템즈가 "확장성 높은 IaC 코드", "Kubernetes 클러스터 프로비저닝 Terraform 모듈"을 우대하는 부분이 정확히 이 결의 일이라 자신 있게 기여할 수 있습니다.

## (3) CS 기본기 위에서 인프라를 디버깅하는 힘

École 42에서 C로 셸(Minishell)과 C++로 IRC 서버를 외부 라이브러리 없이 직접 구현했습니다. fork·pipe·signal·file descriptor를 손으로 다뤄봤기 때문에, k3s에서 컨테이너가 죽었을 때

"OOM Killer가 잡았는지, 시그널이 부모로 안 올라갔는지, 좀비 프로세스가 쌓이는지"를 짐작할 수 있는 감각이 있습니다. IRC 서버에서 poll/select로 멀티플렉싱을 직접 구현했기 때문에, Traefik의 connection draining이나 Kubernetes 서비스 라우팅이 갑자기 이상하게 동작할 때 어디서부터 패킷을 봐야 할지 막막하지 않습니다.

프레임워크가 추상화한 것을 추상화 그대로만 다루는 것과, 그 아래에 무엇이 있는지 알고 다루는 것은 장애 시간에 큰 차이를 만든다고 믿습니다. 인프라 엔지니어에게 CS 기본기는 "있으면 좋은 것"이 아니라 "새벽 3시에 차이를 만드는 것"입니다.

#### (4) AI 도구를 운영 자동화로 끌어오는 실행력

API가 없는 서울시 25개 구 도서관 사이트를 통합하기 위해 LangGraph + Playwright로 자율 에이전트를 만들었습니다. 단순 스크립트가 아니라 "팝업 발생 시 대응", "로그인 우회", "검색 결과 부재 시 대체 도서관 탐색" 같은 판단 노드를 그래프로 설계해 검색 시간을 5분에서 1분으로 줄였습니다. 핵심은 "AI를 신기하게 쓰는 것"이 아니라 "기존 사람이 매번 하던 판단을 그래프 노드로 옮겨 재현 가능하게 만드는 것"이었습니다.

이 사고방식은 운영 자동화에도 그대로 옮겨집니다. 반복되는 운영 의사결정—"이 알람이 진짜 알람인가", "이 배포는 카나리로 갈까 풀 롤아웃으로 갈까"—을 LLM과 도구 호출로 표준화하면, 사람이 한 번 잘 정의한 판단이 시스템에 영구히 남습니다. 데브시스터즈가 우대하는 "AI/LLM을 업무에 잘 활용하는 분"이라는 조건에서 제가 자신 있게 기여할 수 있는 영역입니다.

운영하지 않은 코드는 절반밖에 배운 것이 아니라고 생각합니다. 만들고, 배포하고, 새벽 3시에 책임지고, 그 경험을 다음 사람을 위한 표준으로 남기는 일—이 사이클을 데브시스터즈 인프라셀에서 함께 돌리고 싶습니다.